



[SET 322]: Distributed Computing

Project idea: Distributed To-Do List

Project Name: Task Tracker

Presented by:

Nada Elsayy	192200164
Caren Moheb	192200186
Marvel Fady	192200187
Mohammed Tariq	192200244
Ahmed Ehab	192200147
Fredy George	192200214
Mazen Abdallah	192200149

Presented to:

Dr/ Doaa Mabrouk

TA/ Mohamed Tarek - Mohamed Khattab

I. Introduction & Overview

The **Task Tracker** is a distributed application designed to help users manage personal and shared to-do lists. It operates using a client-server architecture, allowing users to interact with a centralized task database through multiple interfaces, including a web application and a command-line interface (CLI).

The system emphasizes collaboration, thread safety and state persistence, allowing multiple users to sign up, log in, and manage their tasks either in a private "Personal" space or a collaborative "Shared" board.

- **Thread safety:** means that if 2 users are logged into the same "Shared" board at the same time, the code is designed to prevent this crash. In Server.py, we achieved this using (`list_lock = threading.Lock()`).
- **State persistence:** means that if we turn off the server, close the laptop, and turn it back on tomorrow, all users and tasks will still be exactly where as we left them. As Server.py writes all of this data directly into the `todo_db.json` file on the hard drive using (`save_db()` function).

II. System Architecture

The project is built as a **3-Tier Architecture** (when using the web interface) or a **2-Tier Architecture** (when using the CLI client).

- **Data Tier (Server):** The Server.py acts as the single source of truth. It manages the central database (`todo_db.json`), handles authentication, and processes task operations.
- **Application/Logic Tier (Web App):** The WebApp.py serves as a middle-man. It exposes HTTP endpoints and renders HTML pages for browser clients, while internally communicating with the Data Tier via raw TCP sockets.
- **Presentation Tier (Clients):**
 - Web Browser: Interacts with the WebApp.py via HTTP/AJAX.
 - CLI Client: Interacts directly with the Server.py via TCP sockets.

Distributed System Characteristics:

- **Concurrency:** The server's ability to handle multiple users at the exact same time without freezing or slowing down.
- **Synchronization & Thread Safety:** A global `list_lock` (Mutex) is used in the server to prevent race conditions when multiple users attempt to read or write to the JSON database simultaneously.
- **Message Passing:** Communication between the nodes (Web App ↔ Server, CLI ↔ Server) happens via JSON-encoded payloads over TCP sockets.
- **Stateful Storage:** The server maintains persistence by saving state to `todo_db.json`.

III. Codebase Analysis

1. Server.py (The Core Engine & Data Tier)

This file acts as the central brain of the distributed system, handling all data processing, security, and storage without any graphical interface.

Key Responsibilities:

- **Central Database Management:** Reads and writes all user and task data to the persistent `todo_db.json` file.
- **Concurrency Control:** Utilizes threading. Thread to spawn a new process for every connecting user, allowing the server to handle dozens of requests simultaneously.
- **Data Safety (Synchronization):** Implements a Mutex (`list_lock`) to force threads to wait their turn when physically saving data, preventing the JSON file from corrupting during simultaneous saves.
- **Security & Authentication:** Uses the SHA-256 algorithm to securely hash user passwords before saving them, and generates unique encrypted tokens for task IDs.
- **Command Processing:** Parses incoming JSON messages over TCP sockets to execute commands like ADD, DELETE, LOGIN, and GET.

Code:

```
import socket
import threading
import json
import os
import hashlib
import secrets
import time

DB_FILE = "todo_db.json"
list_lock = threading.Lock()

def load_db():
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "r") as f:
            return json.load(f)
    return {"users": {}, "tasks": []}

def save_db(db):
    with open(DB_FILE, "w") as f:
        json.dump(db, f, indent=2)

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def get_visible_tasks(tasks, username, list_type, project_id=None):
```

```

"""Return tasks visible to the user based on their list type and project ID."""
if list_type == "personal":
    # Strictly private: only tasks the user owns that are personal
    return [t for t in tasks
            if t.get("owner") == username and t.get("list_type") == "personal"]
else:
    # Shared: only tasks belonging to the exact same project_id
    return [t for t in tasks
            if t.get("list_type") == "shared" and t.get("project_id") == project_id]

def handle_client(client_socket):
    while True:
        try:
            request_data = client_socket.recv(4096).decode('utf-8')
            if not request_data:
                break

            request = json.loads(request_data)
            command = request.get("command")

            with list_lock:
                db = load_db()
                users = db["users"]
                tasks = db["tasks"]

            # — AUTH COMMANDS —————
            if command == "SIGNUP":
                username = request.get("username")
                password = request.get("password")
                list_type = request.get("list_type", "personal")
                project_id = request.get("project_id")
                project_id = project_id.strip().upper() if project_id else ""
                shared_action = request.get("shared_action", "create")

                if list_type not in ("personal", "shared"):
                    list_type = "personal"

                # Get all existing project IDs from users and tasks
                existing_ids = {u.get("project_id") for u in users.values() if u.get("project_id")}
                existing_ids.update({t.get("project_id") for t in tasks if t.get("project_id")})

                if list_type == "shared" and not project_id:
                    response = {"status": "error", "message": "A Project ID is required for Shared
lists"}

```

```

        elif list_type == "shared" and shared_action == "join" and project_id not in
existing_ids:
            response = {"status": "error", "message": "Project ID not found. Please check the
ID and try again."}
        elif list_type == "shared" and shared_action == "create" and project_id in
existing_ids:
            response = {"status": "error", "message": "Project ID already exists. Please pick
a different one."}
        elif username in users:
            response = {"status": "error", "message": "Username already exists"}
        else:
            user_record = {
                "password": hash_password(password),
                "list_type": list_type,
                "project_id": project_id if list_type == "shared" else None
            }
            users[username] = user_record
            save_db(db)
            response = {
                "status": "success",
                "message": "Account created",
                "list_type": list_type,
                "project_id": project_id if list_type == "shared" else None
            }

elif command == "LOGIN":
    username = request.get("username")
    password = request.get("password")
    user = users.get(username)

    list_type = request.get("list_type", "personal")
    project_id = request.get("project_id")
    project_id = project_id.strip().upper() if project_id else ""
    shared_action = request.get("shared_action", "create")

    if user and user["password"] == hash_password(password):
        if list_type not in ("personal", "shared"):
            list_type = "personal"

        existing_ids = {u.get("project_id") for u in users.values() if
u.get("project_id")}
        existing_ids.update({t.get("project_id") for t in tasks if t.get("project_id")})

        if list_type == "shared" and not project_id:

```

```

        response = {"status": "error", "message": "A Project ID is required for Shared
lists"}

        elif list_type == "shared" and shared_action == "join" and project_id not in
existing_ids:

            response = {"status": "error", "message": "Project ID not found. Please check
the ID and try again."}

        elif list_type == "shared" and shared_action == "create" and project_id in
existing_ids:

            response = {"status": "error", "message": "Project ID already exists. Please
pick a different one."}

        else:

            # CRITICAL FIX: Only update project_id if they log into a shared board.
            # If they log into personal, DO NOT overwrite their project_id with None.
            # This allows the user to have both a personal list and a shared list
simultaneously!

            if list_type == "shared":

                user["project_id"] = project_id

            user["list_type"] = list_type
            save_db(db)

            response = {
                "status": "success",
                "message": "Logged in",
                "list_type": list_type, # Return the list_type they selected on the
login page!

                "project_id": project_id if list_type == "shared" else None
            }

        else:

            response = {"status": "error", "message": "Invalid username or password"}

# — TASK COMMANDS —————
elif command == "GET":

    username = request.get("username")
    list_type = request.get("list_type", "personal")
    project_id = request.get("project_id")
    visible = get_visible_tasks(tasks, username, list_type, project_id)
    response = {"status": "success", "data": visible}

elif command == "ADD":

    username = request.get("username")
    list_type = request.get("list_type", "personal")
    project_id = request.get("project_id")
    task_text = request.get("task")
    priority = request.get("priority", "med")

```

```

task = {
    "id": secrets.token_hex(8),
    "text": task_text,
    "owner": username,
    "list_type": list_type,
    "project_id": project_id if list_type == "shared" else None,
    "priority": priority,
    "done": False,
    "ts": int(time.time())
}
tasks.insert(0, task)
save_db(db)
visible = get_visible_tasks(tasks, username, list_type, project_id)
response = {"status": "success", "data": visible}

elif command == "TOGGLE":
    task_id = request.get("id")
    username = request.get("username")
    list_type = request.get("list_type", "personal")
    project_id = request.get("project_id")
    task = next((t for t in tasks if t["id"] == task_id), None)
    if not task:
        response = {"status": "error", "message": "Task not found"}
    elif task["owner"] != username:
        response = {"status": "error", "message": "Not your task"}
    else:
        task["done"] = not task["done"]
        save_db(db)
        visible = get_visible_tasks(tasks, username, list_type, project_id)
        response = {"status": "success", "data": visible}

elif command == "DELETE":
    task_id = request.get("id")
    username = request.get("username")
    list_type = request.get("list_type", "personal")
    project_id = request.get("project_id")
    task = next((t for t in tasks if t["id"] == task_id), None)
    if not task:
        response = {"status": "error", "message": "Task not found"}
    elif task["owner"] != username:
        response = {"status": "error", "message": "Not your task"}
    else:
        tasks.remove(task)
        save_db(db)
        visible = get_visible_tasks(tasks, username, list_type, project_id)

```

```

        response = {"status": "success", "data": visible}

    elif command == "EDIT":
        task_id = request.get("id")
        new_text = request.get("task")
        username = request.get("username")
        list_type = request.get("list_type", "personal")
        project_id = request.get("project_id")
        task = next((t for t in tasks if t["id"] == task_id), None)
        if not task:
            response = {"status": "error", "message": "Task not found"}
        elif task["owner"] != username:
            response = {"status": "error", "message": "Not your task"}
        else:
            task["text"] = new_text
            save_db(db)
            visible = get_visible_tasks(tasks, username, list_type, project_id)
            response = {"status": "success", "data": visible}

    else:
        response = {"status": "error", "message": "Unknown command"}

    client_socket.send(json.dumps(response).encode('utf-8'))

except Exception as e:
    print(f"Client error: {e}")
    break

client_socket.close()

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("127.0.0.1", 5555))
    server.listen(5)
    print("Server listening on port 5555...")
    while True:
        client, addr = server.accept()
        print(f"Connected: {addr}")
        threading.Thread(target=handle_client, args=(client,)).start()

if __name__ == "__main__":
    start_server()

```


2. WebApp.py (The Application Tier)

This file acts as the "middle-man," bridging the gap between the user's visual web browser and the raw socket-based Server.py.

Key Responsibilities:

- **Frontend Hosting:** Uses the Flask framework to host the web server (on port 8080) and serve the HTML/CSS templates to the user's browser.
- **Protocol Translation:** Catches standard HTTP requests (button clicks on the website) and translates them into raw JSON socket messages that the backend Server.py can understand.
- **Session Management:** Uses Flask session variables to securely remember if a user is logged in, their username, and whether they are viewing a Personal or Shared board.
- **API Endpoints:** Provides asynchronous AJAX routes (/add, /delete, /toggle) so the web page can update task lists dynamically without forcing the user to refresh the browser.

Code:

```
from flask import Flask, render_template, request, jsonify, session, redirect, url_for
import socket
import json

app = Flask(__name__)
app.secret_key = "super_secret_key_change_this"

def talk_to_server(payload):
    try:
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect(("127.0.0.1", 5555))
        client.send(json.dumps(payload).encode('utf-8'))
        response = client.recv(4096).decode('utf-8')
        client.close()
        return json.loads(response)
    except:
        return {"status": "error", "message": "Server offline"}

# — AUTH ROUTES —

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username').strip()
        password = request.form.get('password').strip()
```

```

list_type = request.form.get('list_type', 'personal')
shared_action = request.form.get('shared_action', 'create')
if shared_action == 'create':
    project_id = request.form.get('create_project_id', '').strip().upper()
else:
    project_id = request.form.get('join_project_id', '').strip().upper()

if list_type not in ('personal', 'shared'):
    list_type = 'personal'
if list_type == 'shared' and not project_id:
    return render_template('login.html', error="Please provide a Project ID for the Shared list")

res = talk_to_server({
    "command": "LOGIN",
    "username": username,
    "password": password,
    "list_type": list_type,
    "project_id": project_id if list_type == 'shared' else None,
    "shared_action": shared_action if list_type == 'shared' else None
})
if res["status"] == "success":
    session['username'] = username
    session['list_type'] = res.get("list_type", "personal")
    session['project_id'] = res.get("project_id")
    return redirect(url_for('home'))
return render_template('login.html', error=res["message"])
return render_template('login.html')

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        password = request.form.get('password', '').strip()
        list_type = request.form.get('list_type', 'personal')

        shared_action = request.form.get('shared_action', 'create')
        if shared_action == 'create':
            project_id = request.form.get('create_project_id', '').strip().upper()
        else:
            project_id = request.form.get('join_project_id', '').strip().upper()

        if list_type not in ('personal', 'shared'):
            list_type = 'personal'
        if not username or not password:

```

```

        return render_template('signup.html', error="All fields required")
    if list_type == 'shared' and not project_id:
        return render_template('signup.html', error="Please provide a Project ID for the Shared list")

    res = talk_to_server({
        "command":      "SIGNUP",
        "username":      username,
        "password":      password,
        "list_type":     list_type,
        "project_id":    project_id if list_type == 'shared' else None,
        "shared_action": shared_action if list_type == 'shared' else None
    })
    if res["status"] == "success":
        session['username'] = username
        session['list_type'] = list_type
        session['project_id'] = project_id if list_type == 'shared' else None
        return redirect(url_for('home'))
    return render_template('signup.html', error=res["message"])
return render_template('signup.html')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

# — TASK ROUTES —————

@app.route('/')
def home():
    if 'username' not in session:
        return redirect(url_for('login'))
    list_type = session.get('list_type', 'personal')
    project_id = session.get('project_id')
    data = talk_to_server({
        "command":      "GET",
        "username":      session['username'],
        "list_type":     list_type,
        "project_id":    project_id
    })
    tasks = data.get("data", [])
    return render_template('index.html',
                           tasks=tasks,
                           username=session['username'],
                           list_type=list_type,
                           project_id=project_id)

```

```
@app.route('/add', methods=['POST'])
def add():
    if 'username' not in session:
        return jsonify({"status": "error"}), 401
    task_text = request.json.get('task')
    priority = request.json.get('priority', 'med')
    list_type = session.get('list_type', 'personal')
    project_id = session.get('project_id')
    res = talk_to_server({
        "command": "ADD",
        "username": session['username'],
        "list_type": list_type,
        "project_id": project_id,
        "task": task_text,
        "priority": priority
    })
    return jsonify(res)

@app.route('/toggle', methods=['POST'])
def toggle():
    if 'username' not in session:
        return jsonify({"status": "error"}), 401
    list_type = session.get('list_type', 'personal')
    project_id = session.get('project_id')
    res = talk_to_server({
        "command": "TOGGLE",
        "username": session['username'],
        "list_type": list_type,
        "project_id": project_id,
        "id": request.json.get('id')
    })
    return jsonify(res)

@app.route('/delete', methods=['POST'])
def delete():
    if 'username' not in session:
        return jsonify({"status": "error"}), 401
    list_type = session.get('list_type', 'personal')
    project_id = session.get('project_id')
    res = talk_to_server({
        "command": "DELETE",
        "username": session['username'],
        "list_type": list_type,
        "project_id": project_id,
```

```

        "id":      request.json.get('id')
    })
    return jsonify(res)

@app.route('/edit', methods=['POST'])
def edit():
    if 'username' not in session:
        return jsonify({"status": "error"}), 401
    list_type = session.get('list_type', 'personal')
    project_id = session.get('project_id')
    res = talk_to_server({
        "command":    "EDIT",
        "username":    session['username'],
        "list_type":   list_type,
        "project_id":  project_id,
        "id":          request.json.get('id'),
        "task":        request.json.get('task')
    })
    return jsonify(res)

if __name__ == '__main__':
    app.run(debug=True, port=8080)

```

3. Client.py (The Command-Line Interface)

This file provides a purely text-based method for interacting with the system, demonstrating the flexibility of the distributed architecture.

Key Responsibilities:

- **Direct Communication:** Bypasses the Web App entirely and connects directly to the Server.py backend (on port 5555) using TCP sockets.
- **Raw Message Passing:** Automatically packages the user's text inputs into structured JSON payloads (e.g., {"command": "ADD", "task": "..."}) to communicate with the server.
- **Low-Resource Operation:** Provides a lightweight, terminal-based menu loop that allows users or automated scripts to interact with the database using almost zero computer resources.
- **Proof of Decoupling:** Proves that the system is a true distributed architecture, as the core database server can successfully serve a graphical website and a text-based terminal at the exact same time.

Code:

```
import socket
import json

def send_command(command, task=None):
    try:
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect(("127.0.0.1", 5555))

        payload = {"command": command, "task": task}
        client.send(json.dumps(payload).encode('utf-8'))

        response = client.recv(1024).decode('utf-8')
        res_json = json.loads(response)

        print("\n--- Server Response ---")
        if res_json["status"] == "success":
            print(f"Tasks: {res_json.get('data', [])}")
        else:
            print(f"Error: {res_json.get('message')}")

        client.close()
    except Exception as e:
        print(f"Could not connect to server: {e}")

def main():
    while True:
        print("\n--- Distributed To-Do Menu ---")
        print("1. View Tasks (GET)")
        print("2. Add Task (ADD)")
        print("3. Delete Task (DELETE)")
        print("4. Exit")

        choice = input("Select an option (1-4): ")

        if choice == '1':
            send_command("GET")
        elif choice == '2':
            new_task = input("Enter the task: ")
            send_command("ADD", new_task)
        elif choice == '3':
            del_task = input("Enter the task name to delete: ")
            send_command("DELETE", del_task)
        elif choice == '4':
```

```
        print("Goodbye!")
        break
    else:
        print("Invalid choice, try again.")

if __name__ == "__main__":
    main()
```

4. login.html & signup.html (The Authentication Interface)

These files serve as the "front door" to the application, providing a secure and visually appealing way for users to identify themselves to the system

5. index.html (The Main Dashboard)

This file is the core interface of the application. It acts as an interactive Kanban board where the user actually manages their daily workflow.

IV. What We Achieved as a Distributed System

1. **Single Coherent System from Autonomous Nodes:** We built a distributed system using independent, autonomous computing elements (the backend server, the web application, and the CLI clients). These nodes collaborate over a network but appear to the end-user as a single, seamless Task Tracker application.
2. **Three-Tiered Architecture & Vertical Distribution:** The system successfully demonstrates a multi-tiered centralized architecture. We applied vertical distribution by splitting the application into distinct logical layers running independently: the Presentation layer (Web Browser), the Application server (WebApp.py), and the Database server (Server.py).
3. **Basic Client-Server Model:** We strictly followed the traditional client-server characteristics. The Server.py acts as the process offering services, while the web app and CLI act as the client processes requesting services, communicating through a strict request/reply model over the network.
4. **Resource-Based Interactions:** Inspired by RESTful architecture styles, the system treats user tasks as remote resources. These resources can be created (ADD), retrieved (GET), modified (EDIT/TOGGLE), and removed (DELETE) by remote applications using a standardized, self-describing interface (JSON messages).
5. **System Openness and Interoperability:** By defining a strict, well-defined interface (JSON payloads over TCP sockets), the system achieves openness. It effortlessly interoperates with entirely different types of clients (a graphical Flask web application and a raw Python terminal script) irrespective of their underlying environments.
6. **Thread-Based Concurrency:** To handle multiple users at the exact same time, the server uses lightweight "threads" instead of creating heavy, full-sized computer processes. By running a new threading. Thread for every user that connects, the server can process many requests simultaneously without freezing, saving computer memory and ensuring no user has to wait in line for another user to finish.
7. **Application-Level Middleware:** The WebApp.py file acts as the ultimate middleware. A web browser only speaks HTTP, but Server.py only speaks TCP Sockets and JSON. WebApp.py acts as the translator in the middle. It takes the browser's HTTP click, converts it into a JSON socket message, sends it to the server, and converts the reply back to the browser.
8. **Transient Synchronous Communication:** The project's networking model strictly follows a transient synchronous communication paradigm over the Transport Layer (TCP). When a client application issues a command, it actively blocks and waits for the server to process the request and deliver the reply, requiring both components to be active simultaneously.